Here we consider how a learning to rank (LTR) pipeline work end to end, from data collection to inference.

Data part:

Data collection for learning to rank { explicit feedback / implicit feedback.

Explicit feedback

Scores (e.g. ranks)
$S_{i,1}$
S_{i,k_i}

pros: the dataset is clean and unbiased

Cons: too expensive and cannot scale up easily

Implicit feedback (we focus on implicit feedback because it is more practical)

Rely on actual user data for labeling.

Log different users behavior, such as

1. Impression
   Which item were physically shown to the user.
   e.g. if didn't scroll down to item 7, then it will not be logged as an impression
2. Click
   Which item is clicked
3. Dwell time
   How long a user stay on an item
4. Conversions
   If a user paid for an item

and any more

and more ...

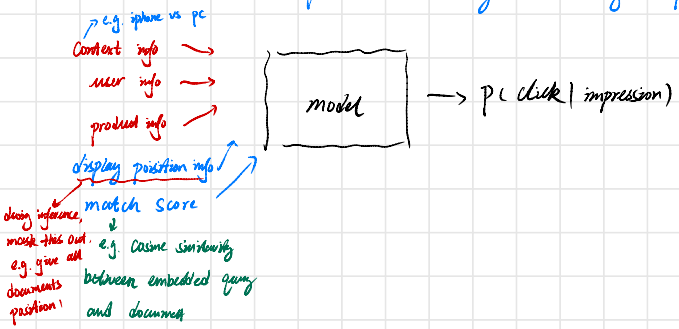
Using the logged info, we can start training models for prediction.

(e.g. Click through rate.
Conversion rate etc.)

Pointwise approach

Click through rate (CTR) model

We can train a model to predict CTR using the items give impression



Similarly, we can train a conversion rate model using the item clicked

→ p(conversion | click)

Then we can compute the score for an item as

Expected revenue = p(click | impression) p(conversion | click) * revenue

↳ from business perspective we care about revenue

probs: get well calibrated probs (not perfect though)

Cons: it ignores the rank (score) between retrieved documents completely → e.g. the loss for

predicted prob	label
0.2	1
0.1	0

is larger than

prob	label
0.5	1
0.51	0

loss is smaller order is wrong

Listwise or pairwise approach

To train listwise or pairwise models we need to score each item.

e.g. if purchase → score 4
add to cart → score 3
click → score 2
unwell → score 1

→ then we can scale the score by the location of each item

e.g. if both item 1 and 2 are clicked, but item 1 is ranked on top, we give item 2 higher score.

pros: it learns from our target (good rank) directly

Cons: the probability is not calibrated
harder to train

Pointwise + Listwise/pairwise loss

probability from the model will be used for both pointwise loss and listwise/pairwise loss

probs: get better probabilities → well calibrated
→ take the score into account

offline metrics for evaluating ranking quality

normalized discounted Cumulative gain (NDCG) → label, e.g. purchase → 4, add to cart → 3, click → 2, dwell → 1

$$DCG_i = \sum_{j=1}^{k_i} \frac{S_{i,j}}{\log_2(j+1)}$$

- $S_{i,j}$ → score of document for query q_i at position j → given by model
- DCG_i → for query i

$iDCG_i$ → DCG_i if the the model ranks perfectly.

ideal DCG

top → bottom

highest score → lowest score

$$NDCG_i = \frac{DCG_i}{iDCG_i}$$